

Attention Has No Sense of Order.

Positional encodings are the trick that gives it one. They also decide whether your model can read 4K tokens or 1M, and why it forgets the middle.

Sinusoidal

RoPE

ALiBi

YaRN

LongRoPE

 16 slides · why order matters · RoPE intuition · how 8K became 128K · lost in the middle



AI Coach John

Follow

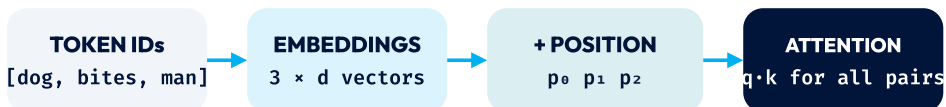


PROITBRIDGE
Strive For Better Future

 **Repost**

Self-attention is a bag of tokens

Attention compares every token with every other token using dot products. Nothing in that math knows which token came first.



Without positions

"dog bites man" and "man bites dog" produce the identical set of attention scores.

With positions

Each token carries "I am 1st / 2nd / 3rd". Order becomes visible to the dot product.

WHY AN AI ENGINEER MUST CARE



Context length

The encoding scheme, not the weights, sets the hard ceiling on how many tokens a model can read.



Context extension

RoPE scaling is how Llama went 4K → 128K and Phi-3 → 2M with only a fraction of pretraining data.



Silent failure

Push past the trained length and perplexity explodes. No error, just garbage output.



Config knobs

`rope_theta` and `rope_scaling` in `config.json` are things you will tune when self-hosting.



Absolute vs relative position

Absolute schemes tag each token with "you are at index 512". Relative schemes only let attention see "you are 7 tokens apart". The second family generalises better.



💡 In 2026 the answer to "which one?" is almost always RoPE. Everything after this slide is about making RoPE reach further.



Sinusoidal and learned positions

Both add a position vector to the token embedding before layer 1. They differ in where that vector comes from.

SINUSOIDAL: A BINARY CLOCK MADE OF SINE WAVES

dim 0 · high frequency



dim d/2 · low frequency



📡 Sinusoidal (2017)

$$PE(pos, 2i) = \sin(pos / 10000^{2i/d}).$$

Every position gets a unique fingerprint. No parameters, works for any length in theory, weak in practice beyond training.

📊 Learned (BERT, GPT-2)

A lookup table: row 0 for position 0, row 1023 for position 1023. Trained like any weight. Position 1024 simply does not exist. Hard wall.

512

BERT positions

1,024

GPT-2 positions

2,048

GPT-3 positions



This is why classic BERT truncates at 512 tokens. It is not a memory limit. The 513th position was never trained.



AI Coach John

Follow

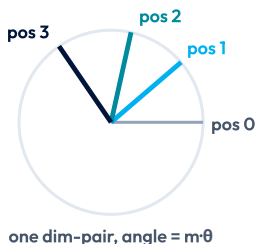


PROTIBRIDGE
Strive For Better Future

↺ Repost

RoPE: encode position as **rotation**

Rotary Position Embedding (Su et al., 2021) does not add anything. It rotates the query and key vectors by an angle proportional to their position.



The key property

$$\text{rot}(q, m) \cdot \text{rot}(k, n) = f(q, k, m - n)$$

the score depends only on the distance $m - n$, never on m or n alone

Many clocks, not one

$d/2$ dimension pairs, each with its own speed $\theta_i = 10000^{(-2i/d)}$.

Fast pairs resolve nearby tokens, slow pairs resolve far apart ones.

↔ Relative for free

Two tokens 7 apart get the same relative signal whether they sit at 10 and 17 or 5,000 and 5,007.

🗄 KV-cache friendly

Rotation is applied once when a key is computed. Cached keys never need re-encoding as the sequence grows.



Nearby tokens end up with similar angles across all pairs, far tokens drift out of phase. Attention naturally decays with distance.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ Repost

RoPE in 15 lines

This is essentially what runs inside every Llama attention layer. Note that it touches q and k only, never v.

```
import torch

def rope_freqs(dim, seq_len, base=10000.0):
    # one angular speed per dimension pair
    inv = 1.0 / (base ** (torch.arange(0, dim, 2) / dim))
    pos = torch.arange(seq_len)
    ang = torch.outer(pos, inv) # [seq, dim/2]
    return torch.polar(torch.ones_like(ang), ang) #  $e^{i \cdot m \cdot \theta}$ 

def apply_rope(x, freqs): # x: [batch, seq, heads, dim]
    xc = torch.view_as_complex(x.float().reshape(*x.shape[:-1], -1, 2))
    return torch.view_as_real(xc * freqs[:, None, :]).flatten(-2).type_as(x)

freqs = rope_freqs(dim=128, seq_len=8192)
q, k = apply_rope(q, freqs), apply_rope(k, freqs) # v untouched
```

[ships RoPE](#)[Llama](#)[Mistral](#)[Qwen](#)[Gemma](#)[DeepSeek](#)[Phi](#)[Falcon](#)[Command R](#)

base is rope_theta in config.json. Llama 2: 10,000. Llama 3: 500,000.
That one number is the story of the next slides.

[AI Coach John](#)[Follow](#)[PROTIBRIDGE](#)
Strive For Better Future[↺ Repost](#)

ALiBi: just penalise distance

Attention with Linear Biases (Press et al., 2021) skips position vectors entirely. It subtracts a distance penalty straight from the attention scores.

ATTENTION SCORE MATRIX · one head, slope $m = 0.5$

$q \cdot k$			
-0.5	$q \cdot k$		
-1.0	-0.5	$q \cdot k$	
-1.5	-1.0	-0.5	$q \cdot k$

$$\text{score} = q \cdot k - m \cdot |i - j|$$

m is fixed per head: $1/2, 1/4, 1/8 \dots 1/256$ for 8 heads.
Steep heads look locally, gentle heads look far.

No learned parameters. No position vectors.
The mask is the same shape at any length.

+ Strengths

Trained at 1K, still sane at 2–4K.
Simple, fast, memory-light. BLOOM-176B and MPT-7B shipped with it.

— Why it lost to RoPE

A fixed penalty cannot express "attend to token 3,000 specifically".
Long-range recall suffers, so most 2024+ LLMs chose RoPE.

 ALiBi lives on in embedding models. Jina and some BGE encoders use it to accept 8K inputs after training on shorter text.



AI Coach John

Follow



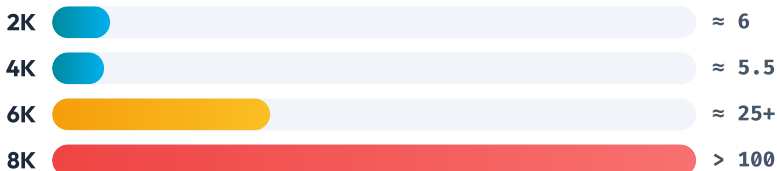
PROITBRIDGE
Strive For Better Future

 Repost

Why 4K models break at 5K

Take Llama 2 (trained at 4,096) and feed it 8,000 tokens. RoPE will happily compute angles for position 7,999. The model has never seen those angles.

PERPLEXITY VS SEQUENCE LENGTH · LLAMA-2-7B, NO SCALING (≈ SHAPE, FROM THE PI PAPER)



🔴 Unseen angles

The slow dimension pairs never completed a full rotation during training. Past 4K they enter territory the weights were never fitted on.

🔥 Attention entropy blows up

With more keys and out-of-distribution scores, softmax flattens. The model attends everywhere and nowhere. Output turns to repetition.

? The question

Can we make positions 0–8K land on angles the model already knows, without retraining from scratch?

✓ The answers

Position Interpolation, NTK-aware scaling, YaRN, LongRoPE. Four ideas, one goal: reuse trained angles.



Position Interpolation: squeeze the ruler

Meta, 2023. Instead of extrapolating to position 8,000, rescale so that 8,000 becomes 4,000. Every new position lands between two old ones.

TRAINED RANGE · 0 ... 4096



INTERPOLATED · 0 ... 8192 mapped into the same range ($m \rightarrow m / 2$)



m / s

position $\div$ scale factor $s =$
 L' / L

≈ 1000

fine-tune steps to recover
quality

32K

Llama 1 extended from 2K in
the paper

+ Why it works

Interpolation between seen angles is a much smaller ask than extrapolation beyond them. The upper bound on error is $\sim 600\times$ tighter.

— The cost

Every dimension is squeezed, including the fast ones. Adjacent tokens become harder to tell apart. Short-context quality drops slightly.



NTK-aware scaling: **change the base**

A Reddit post (bloc97, mid-2023) that became standard practice. Do not scale positions. Scale rope_theta instead, so slow dimensions stretch and fast ones barely move.

Position Interpolation

every pair slowed by the same factor



NTK-aware

fast pairs kept, slow pairs stretched



```
# base' = base * s^(d / (d-2))  for scale factor s
# Llama 2 (d=128): 10,000 -> ≈ 10,000 * 4^(1.016) ≈ 41,000 for 4x length

# Hugging Face config.json · dynamic variant, no fine-tune needed
"rope_scaling": { "type": "dynamic", "factor": 4.0 }

# or just ship a bigger base from the start
"rope_theta": 500000.0          # Llama 3 · Qwen2 uses 1,000,000
```

+ Zero-shot ok

"Dynamic NTK" recomputes the base as the sequence grows. Works without any fine-tuning for ~2x stretch.

— Still not free

The slowest pairs end up slightly extrapolated. Past ~4x you want a few hundred fine-tune steps.



AI Coach John

Follow



PROIBRIDGE
Strive For Better Future

↺ Repost

YaRN and LongRoPE

Both treat frequency bands differently. YaRN hand-designs the rule. LongRoPE searches for it.

METHOD	CORE IDEA	DATA TO ADAPT	SHIPPED IN
YaRN (2023)	NTK-by-parts: leave high freq, interpolate low freq, ramp between. Plus a softmax temperature tweak.	$\approx 0.1\%$ of pretrain tokens	DeepSeek-V2/V3, Qwen2.5 (128K), Mistral variants
LongRoPE (2024)	Evolutionary search for a per-dimension rescale factor, then a two-stage extension.	$\approx 1\text{K}$ fine-tune steps	Phi-3 (128K), research 2M runs
Llama 3 scaling	Frequency-band rule close to YaRN, factor 8 over 8K, base 500K.	$\approx 800\text{B}$ tokens continued pretraining	Llama 3.1 / 3.2 / 3.3

YaRN · what happens to each dimension pair, by wavelength relative to the trained length L

$\lambda \ll L \cdot \text{keep}$

local order preserved

ramp · blend

smooth transition

$\lambda \geq L \cdot \text{interpolate}$

reach the far tokens



YaRN also multiplies attention logits by a small factor ($\approx 0.1 \ln(s) + 1$). Longer contexts flatten softmax, this sharpens it back. Small detail, measurable win.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ Repost

How Llama 3.1 reached 128K

Meta did not train at 128K from day one. They pretrained at 8K, then stretched in stages while checking short-context benchmarks did not regress.



six stages of continued pretraining · gate: short-context evals must hold, needle-in-haystack must pass

```
# meta-llama/Llama-3.1-8B · config.json (excerpt)
"max_position_embeddings": 131072,
"rope_theta": 500000.0,
"rope_scaling": {
  "rope_type": "llama3",
  "factor": 8.0,
  "original_max_position_embeddings": 8192,
  "low_freq_factor": 1.0,
  "high_freq_factor": 4.0
}
```

8K × 8 = 64K base reach

slow pairs: interpolate

fast pairs: leave alone

✓ What to copy

Stage the extension. Measure short tasks after each step. Long context that costs you MMLU points is a bad trade.

⚠ What to remember

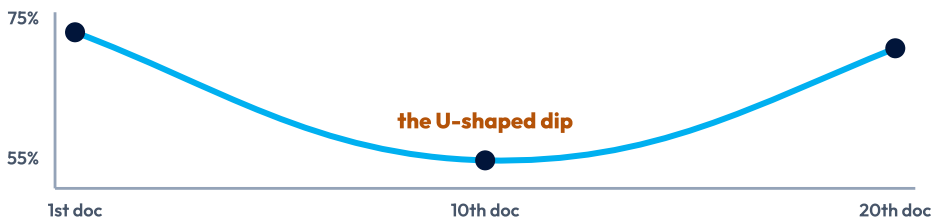
If you load this model with a library that ignores `rope_scaling`, you get the 8K model. Check your inference stack version.



128K claimed $\neq$ 128K usable

"Lost in the Middle" (Liu et al., 2023) showed models recall facts placed at the start or end of the context far better than facts in the middle. Position schemes shape this curve.

ACCURACY vs POSITION OF THE ANSWER · multi-document QA, 20 docs ($\approx$ shape from the paper)



U Why the ends win

Recency: RoPE's distance decay favours nearby tokens. Primacy: early tokens become attention sinks that every layer keeps looking at.

🔪 Needle tests lie a little

A single needle in 128K is easy. RULER and LongBench-style multi-hop tasks show effective length is often half the advertised number.

💡 For RAG: put the most relevant chunk first or last, never buried in the middle of ten others. Rerank, then order deliberately.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ Repost

Long context is never free

The encoding lets the model read 128K. Physics still charges you for it, twice: once in attention compute, once in KV-cache memory.

 $O(n^2)$

attention FLOPs in prefill

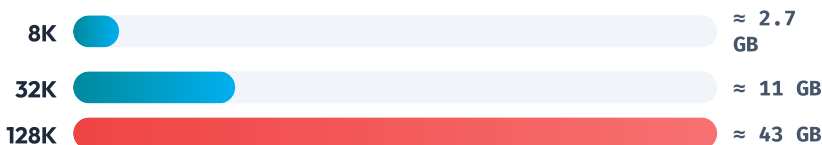
 $O(n)$

KV cache per request

16×

more compute at 4× length

KV CACHE FOR ONE REQUEST · FP16 · LLAMA 3 70B (80 LAYERS, 8 KV HEADS, 128 DIM)



One user, half an H100

43 GB of cache for a single 128K conversation. Batch size collapses, cost per token climbs, time-to-first-token grows to seconds.



What helps

GQA (already assumed above), KV quantization, prefix caching, and honestly: retrieval so you send 8K instead of 128K.



Rule: use long context for documents you cannot chunk well (contracts, code repos, transcripts). Use retrieval for everything else.



AI Coach John

Follow



PROITBRIDGE
Strive For Better Future

↺ Repost

Eight rules for position and length

Everything above, compressed into things you can act on this week.

- 1 Read `max_position_embeddings` and `rope_scaling` before trusting a "128K" label.
- 2 Never exceed the trained length on a model without scaling. Failure is silent.
- 3 Need $\leq 2\times$ more context, no training budget? Dynamic NTK in the inference config.
- 4 Need 4–16 $\times$? YaRN plus a short fine-tune. Check short-context evals after.
- 5 Test with RULER or multi-needle tasks, not a single needle.
- 6 Order RAG context deliberately: best chunk first or last.
- 7 Budget KV memory per user before promising long context in production.
- 8 Upgrade vLLM / TGI / llama.cpp when a new rope type ships or the model degrades to base length.

CONFIG CHEAT CODES YOU WILL SEE

`rope_theta` `rope_scaling.type: linear` `dynamic` `yarn`
`llama3` `longrope` `factor` `original_max_position_embeddings`



Positional encodings on one slide

CONCEPT

REMEMBER

The problem	Attention is permutation-invariant. Positions must be injected or order is invisible.
Sinusoidal	Fixed sin/cos fingerprints added to embeddings. 2017. No params.
Learned	Lookup table per position. BERT 512, GPT-2 1024. Hard wall.
RoPE	Rotate q and k by $\text{position} \times \theta_i$. Scores depend on distance only. Industry default.
ALiBi	Subtract slope $\times$ distance from scores. BLOOM, MPT, some embedders.
Extrapolation	Fails: unseen angles, flat softmax, perplexity explodes past trained length.
Position Interp.	Divide positions by s. ~1K steps fine-tune. Slight short-context loss.
NTK-aware	Raise rope_theta instead. Keeps fast dims. Dynamic variant works zero-shot to $\sim 2\times$.
YaRN	Per-frequency-band blend + logit temperature. ~0.1% data. DeepSeek, Qwen.
LongRoPE	Searched per-dim factors. Phi-3 128K, 2M research runs.
Llama 3.1	8K $\rightarrow$ 128K in 6 stages, base 500K, factor 8, $\approx$ 800B tokens.
Lost in middle	U-shaped recall. Effective length $\approx$ half of advertised. Order RAG chunks.
Cost	$O(n^2)$ compute, $O(n)$ KV. 128K on 70B $\approx$ 43 GB cache per request.



Position is a rotation. Context extension is choosing which rotations to reuse.



AI Coach John

Follow



PROTIBRIDGE
Strive For Better Future

 Repost